

Permission Hijacking at Scale

Permission Hijacking at Scale - Alberto Fernandez-de-Retana, @wowchemy, bubu.

- Published: 2025-07-21
- Original: <<https://albertofdr.github.io/post/permission-hijacking-2025/>>
- Preserved from: <https://albertofdr.github.io/post/permission-hijacking-2025/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Permission Hijacking at Scale

Jul 21, 2025 8 min read **[EN](#)

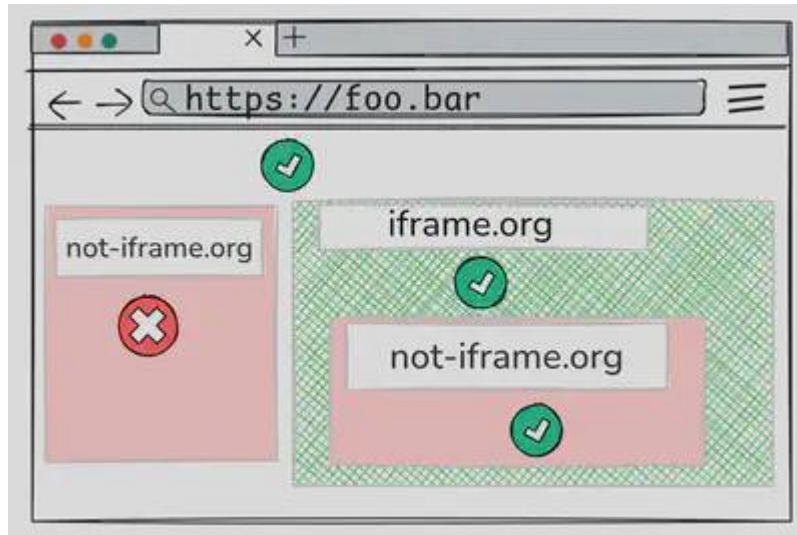
In this blog post, I will explain how to exploit permission hijacking on a large scale. To demonstrate this idea, I will use two companies (LiveChat and Glassix) where I discovered security issues as proof-of-concept (PoC). This blog post is based on research that, if the reviewers are kind, will eventually make its way into an academic conference. Those who know, know.

Here I won't cover the basics or a more general approach. Instead, I'll focus on key concepts, related to the specific threat model we are exploiting. If you're looking for a general introduction, including what is explained here, check out my [blog post on browser permissions in the web security class](#).

Let's start with a few common misconceptions developers have about how browser permissions actually work. After that, I'll walk through the threat model we're exploiting, which shares similarities with supply chain attacks.

Browser Permission Misconception

Misconception I: Restricting Permission Delegation vs Same-Origin-Policy (SOP)



The first misconception is the idea that you can restrict the delegation of permissions at any context as a website developer. For example (see image), suppose your page includes two iframes: `iframe.org` and `not-iframe.org`. You might use the Permissions-Policy header to restrict delegation only to `iframe.org`. However, once the permission is granted to `iframe.org`, the top-level document no longer has control over how that permission is used or further delegated by that origin. As a result, `not-iframe.org` could still gain access to the permission through delegation from `iframe.org`.

Misconception II: Misleading Prompt

[image: misconception]

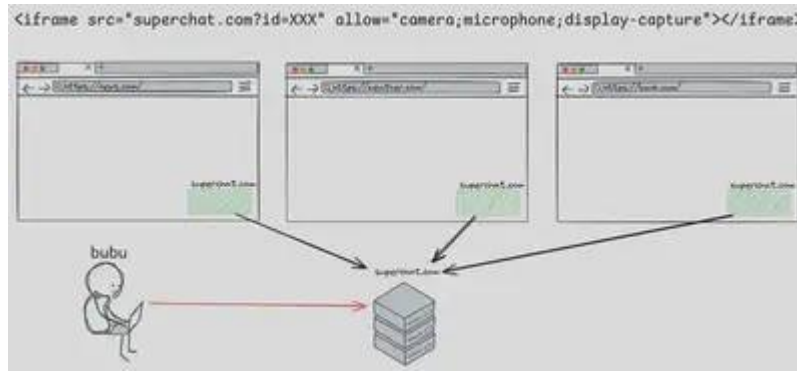
The second misconception is about the permission request prompt. Some developers assume that if an iframe (see Picture), such as `meet.example.org` in the example, requests a permission, the prompt will clearly indicate the origin making the request. In reality, with the exception of a few permissions like `storage-access`, the prompt typically attributes the request to the top-level or currently visited website (`foo.bar`). This can mislead users into believing the main page is requesting the permission, even when it originates from an embedded iframe.

Misconception III: re-Prompt

[image: misconception]

The third misconception concerns reprompting. Using the example in the image, a user grants camera and microphone permissions to a website `foo.bar` for some functionality, such as video conferencing. Later, if an iframe `meet.example.org` is included on the page and inherits these delegated permissions, the iframe will not need to ask the user again for permission and will have access immediately.

Permission Hijacking Through Embedded Documents

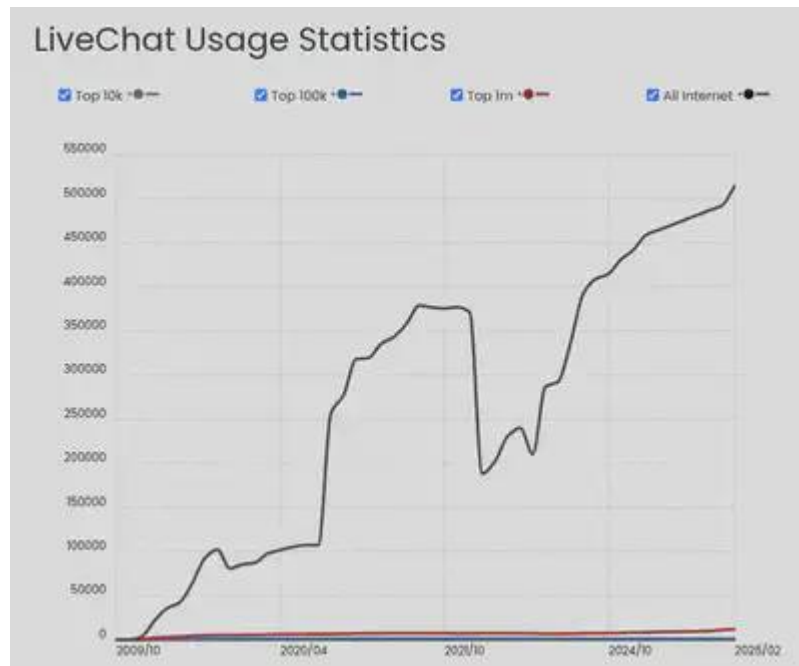


The idea is closely related to supply chain attacks, but it targets a much more specific scenario. As shown in the image, the goal is to compromise a widely embedded document that has been included with delegated permissions. Targeting a widely used support widget allows us to reach more websites and, as a result, more users. To illustrate this with a rough estimate, consider the following hypothesis: if the chat widget is embedded in 5.000 websites, and each site receives 1.000 visitors, that results in up to 5.000.000 potential permission hijacks. Of course, this assumes that users have already granted the relevant permissions or will grant them when prompted. This is where Misconceptions II and III become relevant. In this article, we will not go into possible browser-level mitigations, such as requiring user interaction with the embedded document before permission use/request, limiting permission duration, or other related defenses.



If you are wondering why I specifically mention chat widgets as the target, the full reasoning is detailed in the academic paper. In short, this type of embedded document is extremely widespread and often delegates powerful permissions, even when they are not strictly necessary. In both cases, the idea is the same: gain access to the company's account and inject our payload into the widget.

To give an idea of how widespread this widget is, [PublicWWW](#) reports between 100.000 and 200.000 occurrences depending on the search term, [Wappalyzer](#) estimates around 38,000, and the following graphic comes from [BuiltWith](#):



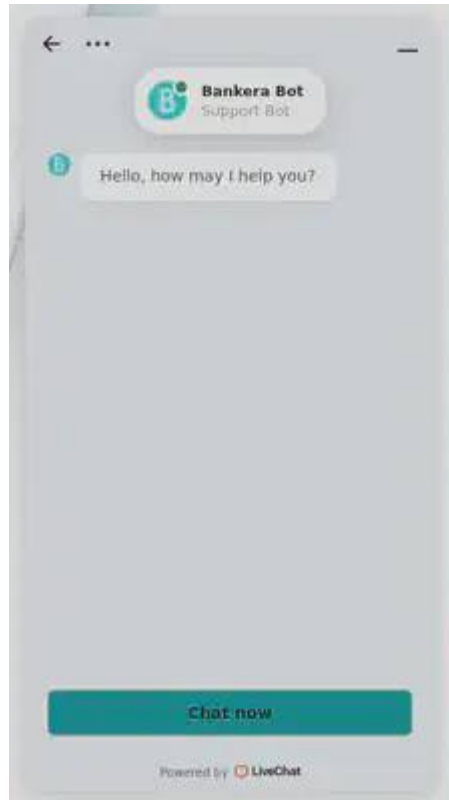
Before we dive in, there's a technical detail you need to know. LiveChat uses two different iframes: the first appears when the chat is minimized and does not use a specific origin, while the second is loaded when the chat is opened and includes a LiveChat origin.

[image: livechat_minimized]

```
<iframe id="chat-widget-minimized" name="chat-widget-minimized" title="LiveChat chat widget" scrolling="no" style="...">
```

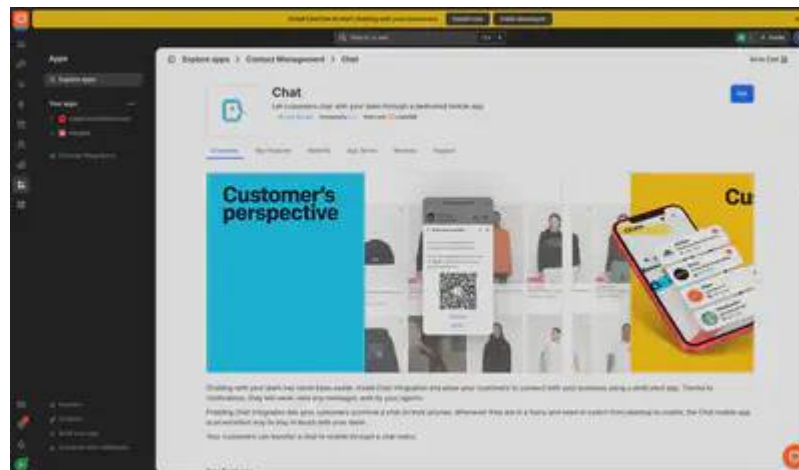
And as mentioned, a second iframe is loaded when the chat is opened, this one using a LiveChat origin. It's important to note that, as far as I can tell, LiveChat changed the default delegation recently, and this delegation is no longer the default.

```
<iframe allow="clipboard-read; clipboard-write; autoplay; microphone *; camera *; display-capture *; picture-in-picture *; fullscreen *;" src="https://secure.livechatinc.com/customer/action/open_chat?..." id="chat-widget" name="chat-widget" title="LiveChat chat widget" scrolling="no" style="...">
```



While thinking about how to breach the widget, and after testing various chat messages, I noticed that the marketplace was integrated into the admin panel.

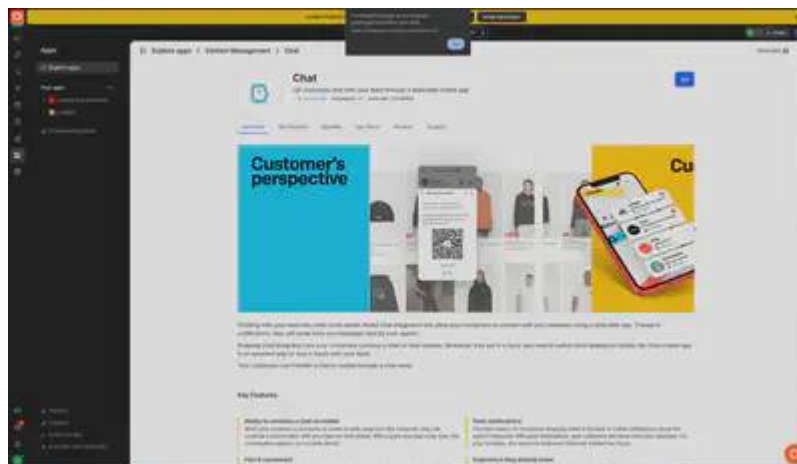
```
<iframe src="https://marketplace-agentapp.livechatinc.com/apps">...</iframe>
```



I noticed the comment section and started testing random inputs. That's when I realized they were using Markdown for comments. Wtf!! I was confused, why would anyone need Markdown in a simple rating comment field?



They were using the `markdown-to-jsx` library. I noticed that previous versions had security issues, and the README did not explicitly mention that the library was secure. So I decided to run a few tests. I found that in some cases, even basic HTML was breaking the marketplace. Style injection was possible, but there was nothing useful to exfiltrate. Meta redirect was also possible, still nothing interesting, maybe phishing. After trying some common XSS techniques, I eventually discovered a one-click XSS using the button `formaction` attribute ([Github Issue](#)). (After my research, other researchers discovered additional security issues in the library that I had not found because they did not work within the comment section). By combining `formaction` with style injection, we could hide the malicious button, make it cover the entire screen, or position it directly over another interface element, such as the plugin installation button.



And you might be thinking, why would XSS in the marketplace matter if it runs on a different origin? Good question. Here is the catch. The key value of the user in that origin was not limited to plugin installation. It granted full access, effectively allowing a complete account takeover. With that, the first part of breaching the company's widget setup was complete.

For the second part, inserting our code into the company's support widget, I moved on to testing the customization features of the chat widgets. Here, I tested all the parameters, and all of them were sanitized except one. And that is all it takes. One unsanitized parameter. That was our entry point. The funny part is that I initially thought the parameter was inside the iframe with a different origin, but it turned out the injection happened in minimized iframe, thus, in the top-level website. That made the attack even worse.

LiveChat Fullchain

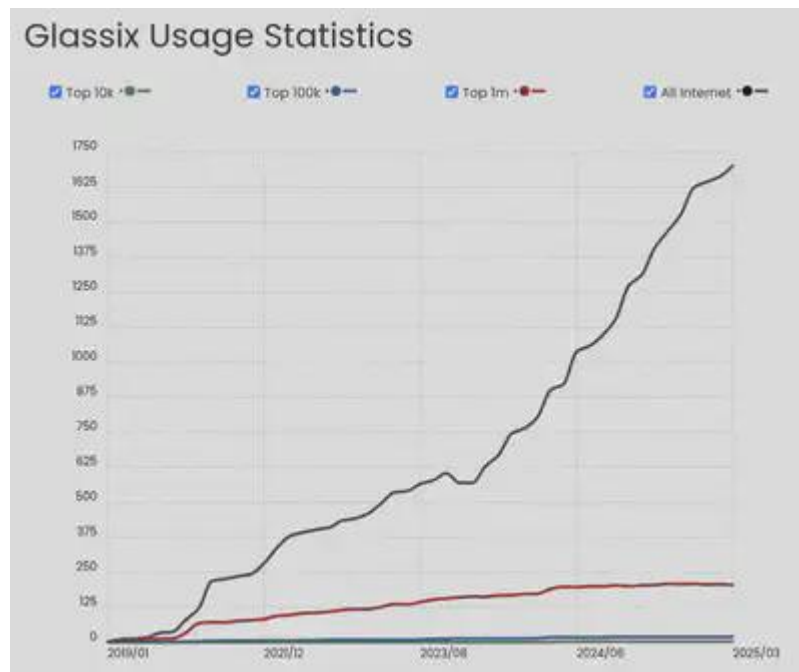
- Comment all the plugins using our one-click XSS with an invisible gigantic button.

- Use the injection to deliver our second payload into the company's support widget.

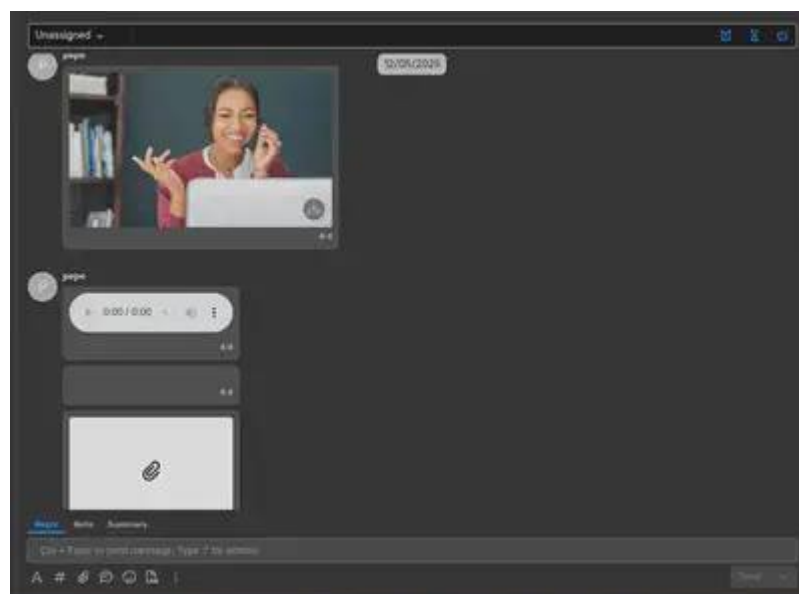
LiveChat Timeline

- 29 nov 2024 - Reported via Email
- 3 dec 2024 - First Reply
- 12 dec 2024 - Fix applied

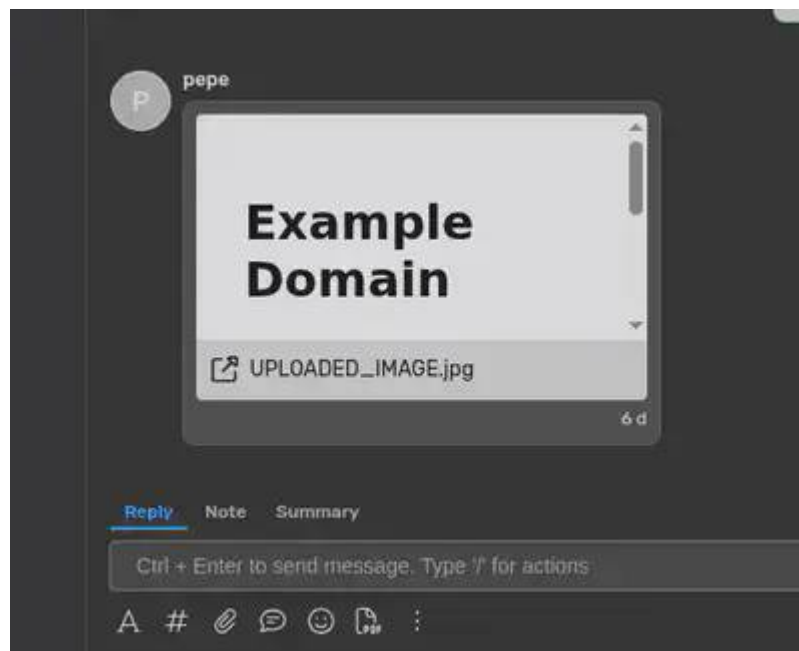
To illustrate its reach, PublicWWW returns 128 results, Wappalyzer estimates 300 active sites, and the following data is sourced from BuiltWith.



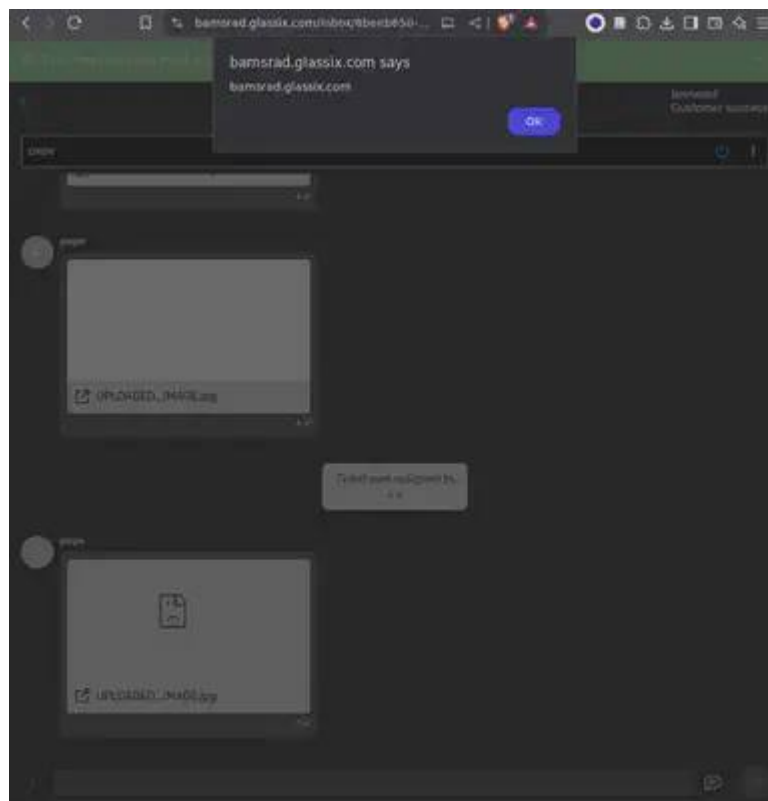
In this second case, I did not even need to dig that deep. I tested how the chat messages were being sent and noticed there was a type field in the message when a file was shared. Thinking with a curious mindset, I wondered what would happen if I changed that value. The following screenshot shows the result of that test.



After trying a few different message types I found that type number 8 rendered an iframe. So I thought what if we use a local-scheme document to trigger XSS? And just like that we had XSS.



But then no alert. Why? They had a Content Security Policy in place. No way. Who even uses CSP? Life just keeps getting less fun. Anyway, it took me five minutes to find a well-known bypass using an Angular endpoint from a Google domain that was whitelisted in the CSP. Thanks [CSPBypass](#).



After gaining XSS, I went straight to testing the chat widget customization. In this case, I did not find anything useful. Then I discovered that the chat widget allowed embedded app installations. I noticed it was possible to install an iframe, so I used the same trick and achieved HTML injection directly into the chat widget. It is true that the user needed to click on the minimized widget to

load our injection and enable the permission hijacking. Still, that was a second permission hijacking discovered.



Glassix Fullchain

- Interact with the company's chat by sending a message that uses a local-scheme document to create an iframe, along with a known `google.com` endpoint inserting Angular to bypass the Content Security Policy.
- Create and install a plugin on the company's support chat that uses a local-scheme document to perform an HTML injection, embedding our iframe with delegated permissions.

Glassix Timeline

- 14 may 2025 - Reported via Email
- 15 may 2025 - First Reply
- 17 may 2025 - Fix applied

Conclusion

These two cases support the idea that permission hijacking is a real threat. Externalized functionality, such as chat widgets running with powerful delegated permissions, makes them a valuable target for malicious actors.

For more general information about browser permissions, I recommend reading [my previous blog post](#). I will also share an update if my paper gets accepted in the future.

Thanks for reading!

